

```
#creating linkedlist
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3
head = n1

while head != None:
    print(head.data, end = "->")
    head = head.next
print("None")
```

10->100->1000->None

```
#sum of values in ll
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1

sum = 0
while head != None:
    sum = sum + head.data
    head = head.next
print(sum)
```

1110

```
#product of values in ll
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1

prod = 1
while head != None:
    prod = prod * head.data
    head = head.next
print(prod)
```

1000000

```
#count no. of nodes in ll
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
```

```

n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1
count = 0
while head != None:
    count = count + 1
    head = head.next
print(count)

```

3

```

#search a target
class linkedlist():
    def __init__(self,data):
        self.data = data
        self.next = None

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.next = n2
n2.next = n3

head = n1
target = int(input())
while head != None:
    if head.data == target:
        print(f"found! The next value is {head.next.data}")
        break
    head = head.next
else:
    print("not found")

```

```

10
found! The next value is 100

```

```

#node update
class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None
    def update(self, data):
        self.data = data

n1 = linkedlist(10)
n2 = linkedlist(100)
n3 = linkedlist(1000)

```

```

n2.update(99)
n1.next = n2
n2.next = n3

head = n1

while head != None:
    print(head.data, end = "->")
    head = head.next
print("None")

```

10->99->1000->None

```

class linkedlist():
    def __init__(self, data):
        self.data = data
        self.next = None
    def insert(self, data):
        new_node = linkedlist(data)
        new_node.next = self.next
        self.next = new_node

n1 = linkedlist(10)

```

```

n2 = linkedlist(100)
n3 = linkedlist(1000)

n1.insert(99)
n1.next.next = n2
n2.next = n3

head = n1
while head != None:
    print(head.data, end = "->")
    head = head.next
print("None")

```

10->99->100->1000->None

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = Node(10)
n2 = Node(100)
n3 = Node(40)
n4 = Node(5)

n1.next = n3
n3.next = n4
n2.next = n4

def insert_at_head(head, data):
    new_node = Node(data)
    new_node.next = head
    return new_node

head = n1
head = insert_at_head(head, 99)
while head != None:
    print(head.data, end = "->")
    head = head.next
print("None")

```

99->10->40->5->None

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

n1 = Node(10)
n2 = Node(100)
n3 = Node(40)
n4 = Node(5)

n1.next = n3
n3.next = n4
n2.next = n4

def insert_at_tail(head, data):
    new_node = Node(data)
    if head == None:
        return new_node
    temp = head
    while temp.next != None:
        temp = temp.next
    temp.next = new_node
    return head

head = n1
head = insert_at_tail(head, 99)
while head != None:
    print(head.data, end = "->")
    head = head.next
print("None")

```

10->40->5->99->None

```
def insert_at_pos(head, data, pos):
    if pos == 0:
        return insert_at_head(head, data)
    new_node = Node(data)
    temp = head
    for i in range(pos-1):
        temp = temp.next
    new_node.next = temp.next
    temp.next = new_node
    return head
```

#Your task is to calculate $ab \bmod 1337$ where a is a positive integer and b is an extremely large positive integer given in the

```
class solution(object):
    def solve(self, a, b):
        """
        :type a: int
        :type b: List[int]
        :rtype: int
        """
        return pow(a, int("".join(map(str, b))), 1337)
```

```
class Solution(object):
    def reverseList(self, head):
        prev = None
        curr = head
        while curr != None:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next
        return prev
```